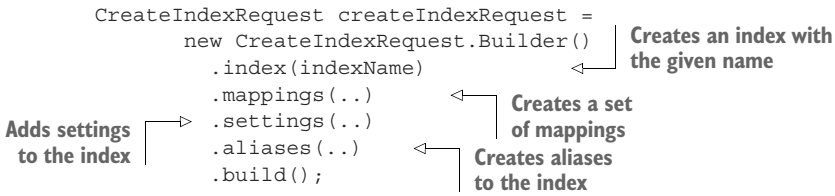
As an index can be created using a schema, with settings as well as aliases, we can chain those methods in the request:



Next, let's see how to index a flight document into our index.

C.7 Indexing documents

In chapter 13, we discussed indexing documents using Query DSL in the Kibana Console (or cURL). For example, this code indexes the flight document with a random ID as the primary key.

Listing C.9 Indexing a document using Query DSL

```
POST flights/_doc
{
  "route":"London to New York",
  "name":"BA123",
  "airline":"British Airways",
  "duration_hours":5
}
```

Here, we invoke a `_doc` endpoint using the `POST` method with document details enclosed as a JSON object. The document ID is generated automatically by Elasticsearch. We can retrieve this document by issuing `GET flights/_doc/_search`.

Let's index the same document, but this time using the Java API client. Remember, we created the `flights` index in the previous section, so all we need to do is use the fluent API to build the query.

Listing C.10 Indexing a document with the Java API client

```
public void indexDocument(String indexName, Flight flight) throws IOException
     {
  IndexResponse indexResponse = this.elasticsearchClient.index(
    i -> i.index(indexName)
          .document(flight)
   );
   System.out.println("Document indexed successfully"+indexResponse);
}
```

Executing this query indexes a flight into our `flights` index. The `elasticsearchClient` exposes an `index` method that can be combined with other methods such as `id` and `document`. In this case, we don't use the ID because we let the system generate it.